



Week 04 Task-11

How the Web works + Burp Suite Intro

CandidateName: Asad Ali

Reference Number: TBS-INT-2026-445

Submission Date: 10|9|2026

Submitted To: TechBiz Security

Table of Contents

1) Executive Summary	3
2) Scope of Task	3
3) Tools	3
4) What is HTTP and HTTPS?	3
4.1) Hyper Text Transfer Protocol (HTTP)	3
4.2) Hyper Text Transfer Protocol Secure (HTTPS)	3
4.2.1) Benefits of HTTPS	4
5) HTTP Request and Response	4
5.1) HTTP Request	4
Example Format of HTTP Request	4
5.2) HTTP Response	4
Example Format of HTTP Response	5
6) HTTP Methods (GET and POST)	5
6.1) GET Method	5
Example Format of GET Request	5
Uses of GET Request:	5
6.2) POST Method	5
Example Format of POST Request	5
Uses of POST Request:	5
6.3) Difference between GET and POST	6
7) HTTP Status Codes	6
7.1) Common HTTP Status Codes	6
7.2) Importance of HTTP Status Codes	7
8) Cookies and Sessions	7
8.1) Cookies	7
Uses of Cookies	7
Example of Cookies	7
8.2) Sessions	7
Uses of Sessions	7
Example of Sessions	7
8.3) Difference between Cookies and Sessions	8
8.4) Importance of Cookies and Sessions	8
9) Burp suite Introduction	8
9.1) Important Features of Burp Suite	8
9.1.1) Proxy	8
9.1.2) Interceptor	9
9.1.3) Repeater	9
9.1.4) Intruder	9
9.1.5) Decoder	9
9.2) How Burp Suite works	9
Without Burp suite	9
With Burp suite	10
9.3) Importance of Burp suite	10
9.4) Launching Burp suite	10
9.5) Configuring FireFox Proxy	11
9.6) Enabling Burp suite Proxy	13
9.7) Intercepting HTTP Request	13
9.7) Modifying HTTP Request	14
10) Conclusion	15

1) Executive Summary

This task focused on building foundational knowledge of web application communication and security testing concepts rather than performing practical penetration testing or exploitation. The study covered HTTP and HTTPS, their benefits and differences, HTTP request and response structure, common HTTP methods such as GET and POST, HTTP status codes, and the role of cookies and sessions in web applications. The task also introduced Burp Suite as a web application security testing platform, including key modules such as Proxy, Intercept, Intruder, Repeater, and Decoder. Overall, the task established a fundamental understanding of web communication and provided an introduction to Burp suite tool and workflow commonly used in web application security assessments.

2) Scope of Task

The scope of this task was to study fundamental web application concepts and understand how web clients and servers communicate using HTTP and HTTPS. It covered HTTP requests and responses, GET and POST methods, status codes, cookies, sessions, and the differences between these concepts. The task also included an introduction to Burp Suite and its key modules, including Proxy, Intercept, Intruder, Repeater, and Decoder.

3) Tools

Burp Suite

4) What is HTTP and HTTPS?

4.1) Hyper Text Transfer Protocol (HTTP)

HTTP stands for **Hyper text transfer protocol**. A communication protocol used between a web browser and web server. When a user visits a website the browser sends an HTTP request to the web server and a server response with a http response the requested web page that user request.

4.2) Hyper Text Transfer Protocol Secure (HTTPS)

HTTPS stands for **Hyper text transfer protocol secure**. It is a secure version of http. It uses SSL (**Secure Socket Layer**) / TLS (**Transport Layer security**) protocol to protect data while it is being transmitted from browser to server basically all data is **encrypted** on the way using **cryptography** and gets **decrypted** only when request reach to the web server. This helps to keep sensitive data secure example. PII (**Personaly identifiable information**), Financial data etc.

4.2.1) Benefits of HTTPS

- Data remains secure from browser to server and server to browser.
- Data integrity prevents data from being tampered by anyone while moving across the network.
- It develops a user trust showing a lock icon the website shows that website is safe to visit.

5) HTTP Request and Response

5.1) HTTP Request

An **HTTP Request** is a message sent by a **web browser** to the **web server** requesting a particular resource or performing an action. **Example.** When user is performing role based login then there will be a request to server that allow the user based on role, username/ email and password.

An HTTP request may contain

- **GET** and **POST** Method
- Header
- URL
- Body

Example Format of HTTP Request

```
GET /index.php HTTP/1.1
Host:192.168.1.100
User-Agent: Mozilla Firefox
Accept: text/html
Content-Length: 45
```

5.2) HTTP Response

An HTTP Response is just like a answer of message that means the server respond to the browser according to query. The server will return a resource that user requested from web server.

An HTTP response may contain

- Status Code
- Headers
- Response Body

Example Format of HTTP Response

```
HTTP/1.1 200 ok
Date: Mon, 10 SEP 2026 05:30:30 UMT
Content-Type: text/html
```

6) HTTP Methods (GET and POST)

6.1) GET Method

The GET method is used to request or retrieve data from a server. The data is usually displayed in the URL **leaking sensitive** information example **email** and **password**.

Example Format of GET Request

```
GET /login?email=asad@gmail.com&password=123456 HTTP/1.1
Host: Techbiz-sec.com
```

Uses of GET Request:

- Searching information
- Retrieving data from a server
- Accessing web pages or resources
- Passing non-sensitive parameters to a server

6.2) POST Method

The **POST** method is used to send data to a server for processing or submission. Unlike GET, the data is normally placed inside the **request body** rather than being displayed in the URL. POST is commonly used when submitting forms, login credentials, or other data that should not appear in the URL.

Example Format of POST Request

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

email=asad@gmail.com&password=123456
```

Uses of POST Request:

- Submitting login credentials.
- Sending form data to a server.

- Creating new records or resources.
- Uploading or submitting data for processing.

6.3) Difference between GET and POST

S.NO	GET	POST
1	Data is sent in url as Query parameters.	Data is not visible in url. It will be in request body.
2	Used to retrieve data.	Used to enter data in database or send data.
3	Data using GET method can be cached	Data using POST method cannot be cached.
4	Pentesting focus is on Parameter manipulation, IDOR/BOLA, injection.	Pentesting focus is on Authentication bypass, injection, parameter tampering, CSRF, file upload issues.

7) HTTP Status Codes

HTTP Status Codes are three-digit codes returned by a web server to indicate the result of an HTTP request. They are used to understand whether a request was successful, redirected, or encountered an error.

7.1) Common HTTP Status Codes

S.No	Status code	Meaning	Description
1	200	ok	Successful request server returned the request data.
2	201	created	Request successful and a new resource has been created
3	301	Moved Permanently	The requested resource has been permanently moved to a new URL.
4	302	Moved Temporarily	The requested resource is temporarily available at a different URL.
5	403	Forbidden	The server understood the request but refuses to authorize or allow access to the requested resource.
6	404	Not Found	The server could not find the requested resource or the resource does not exist.
7	500	Internal Server Error	The server encountered an unexpected condition that prevented it from fulfilling the request.
8	503	Service unavailable	The server is currently unable to handle the request, usually because it is overloaded or temporarily unavailable.
9	504	Gateway Timeout	A server acting as a gateway did not receive a timely response from server.

7.2) Importance of HTTP Status Codes

HTTP status codes help identify whether a web request was successful, redirected, or resulted in an error. During penetration testing, they help security testers understand application behavior and identify potential access-control or server-side issues.

8) Cookies and Sessions

8.1) Cookies

Cookies are small pieces of data stored by a user's browser by a website to remember information about the user. They are used to maintain login states, store user preferences, and help websites recognize users across different requests.

Uses of Cookies

- Storing User preferences.
- Tracking User activity.
- Personalizing website content.

Example of Cookies

```
Set-Cookie: session_id=abc123; Secure; HttpOnly
```

8.2) Sessions

A **session** is a mechanism used by a web application to store and maintain information about a user across multiple HTTP requests. Sessions commonly use a unique **session ID** stored in a cookie to identify the user's session on the server.

Uses of Sessions

- Maintaining User authentication.
- Storing temporary User Data.
- Managing User activity.

Example of Sessions

```
Session ID: S12345  
Username: Riyaz Afridi  
Cart Items: Laptop, Mouse  
Login Status: Authenticated
```

8.3) Difference between Cookies and Sessions

S.NO	Cookies	Sessions
1	Stored primarily in client browsers.	Stored primarily on servers
2	Can be modified by user	Server side sessions can't be directly modified by user.
3	Common purpose is store or send client-side state or identifiers.	Maintains user state.
4	Pentesting concern is on cookie manipulation, theft, weak attributes.	Pentesting concern is on Session fixation, hijacking, invalidation issues.

8.4) Importance of Cookies and Sessions

Cookies and sessions are important for maintaining user state, authentication, and preferences across multiple HTTP requests. In penetration testing, they are examined to identify security issues such as **session hijacking, session fixation, and insecure cookie configurations.**

9) Burp suite Introduction

Burp suite is a web application security testing tool developed by port swigger. It has 2 versions **community** and the **professional** version. It works mainly by acting as an intercepting proxy between **our browser** and the **target web server**. It allow us to capture, inspect, modify, replay and analyze HTTP and HTTPS (**using CA certificate**) request and responses.

9.1) Important Features of Burp Suite

Main Fetures of Burp suite are explained below

- Proxy
- Interceptor
- Repeater
- Intruder
- Decoder

9.1.1) Proxy

Proxy module allow us to intercept HTTP **Requests** and **Responses** between the browser and the web server. It helps in analyzing and modifying web traffic.

9.1.2) Interceptor

Interceptor is a submodule of proxy enabling it allow us to **pauses** any HTTP requests or responses that passes through the proxy. This allow us to view, modify headers, parameters or body data before forwarding it to the destination server. It is mainly used for manual testing.

9.1.3) Repeater

Repeater modules allows manual **manipulation** and repeated **transmission** of individual HTTP requests. It is used to test the impace of modified inputs, perform logic flaws testing and verify the result of vulnerabilities by sending customized payloads multiple time.

9.1.4) Intruder

Intruder module allow us to automate the testing of web application. It allows us to launch **brute-force**, **fuzzing** and payload based attacks by sending multiple requests with different inputs.

9.1.5) Decoder

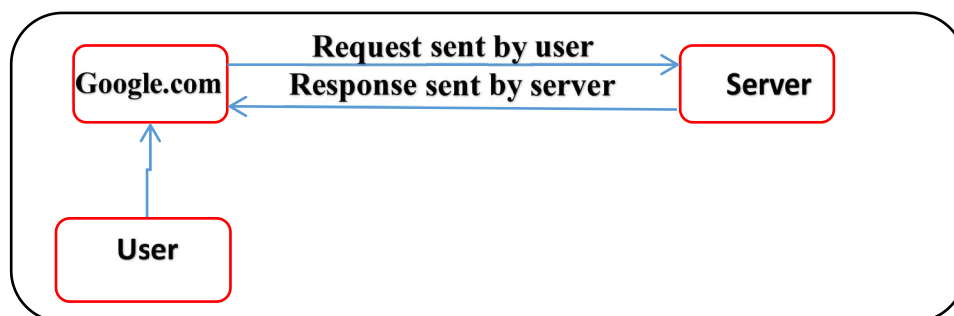
Decoder module help us to **encode data** (converting readable form data into a different form that is not understood by any other user until it decode) and **decode data** (decoding means converting data to its original form that can be easily understand by normal user). It supports encoding and decoding formats like **URL**, **Base64**, **HTML** and **Hex**. It is essential for analyzing tokens, cookies and obfuscated parameters during testing

9.2) How Burp Suite works

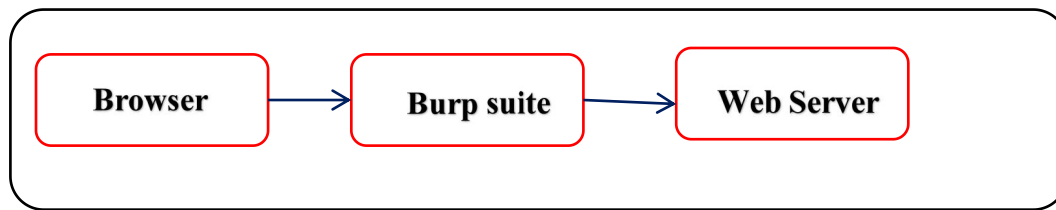
Burpsuite works like a middle main explained in example below.

Without Burp suite

Without burp-suite request flow is simple



With Burp suite

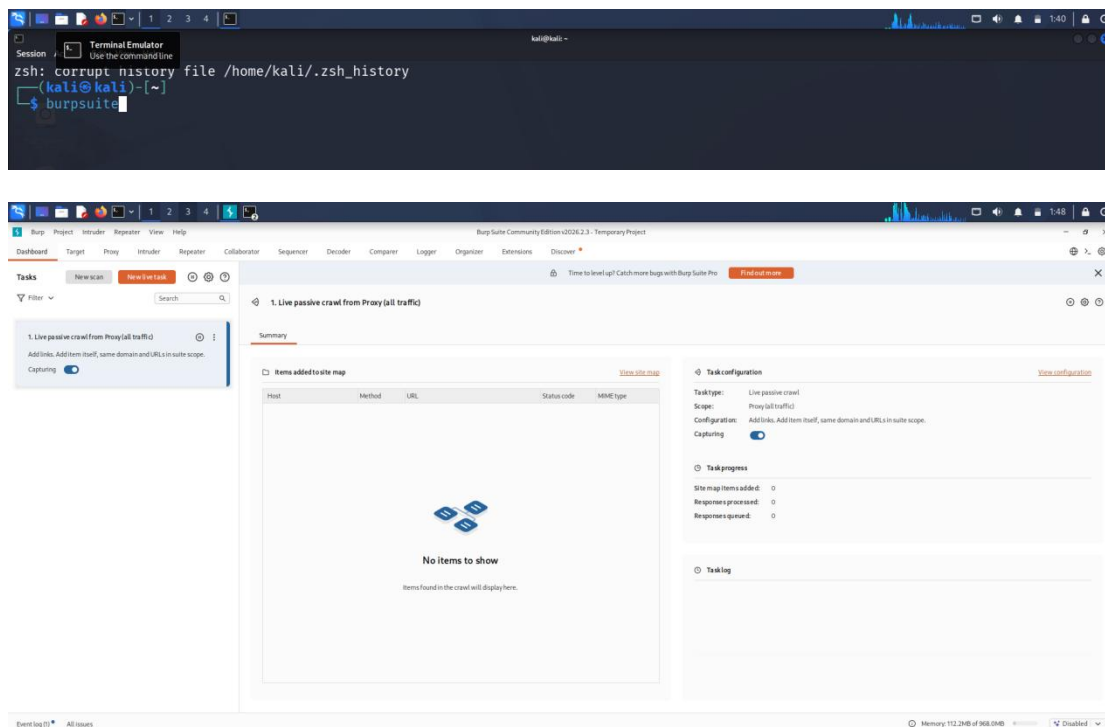


9.3) Importance of Burp suite

Burp Suite is important for web security testing because it allows testers to **intercept, inspect, modify, and analyze HTTP requests and responses**. It helps identify and verify web application vulnerabilities by providing tools for **manual testing, automated testing, and traffic analysis**.

9.4) Launching Burp suite

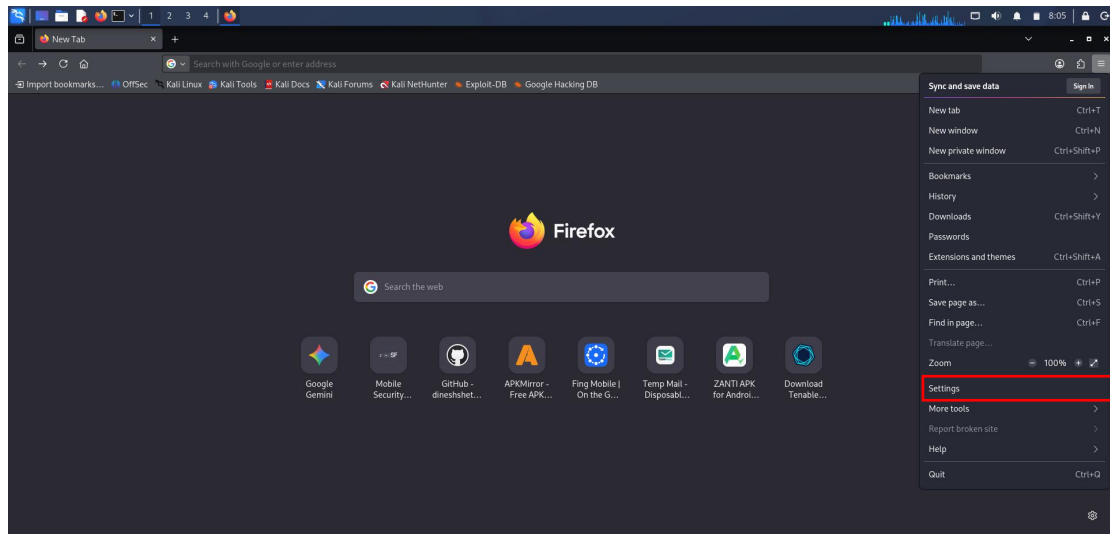
Burp Suite Community edition was launched from kali linux terminal by writing a command **burpsuite**.



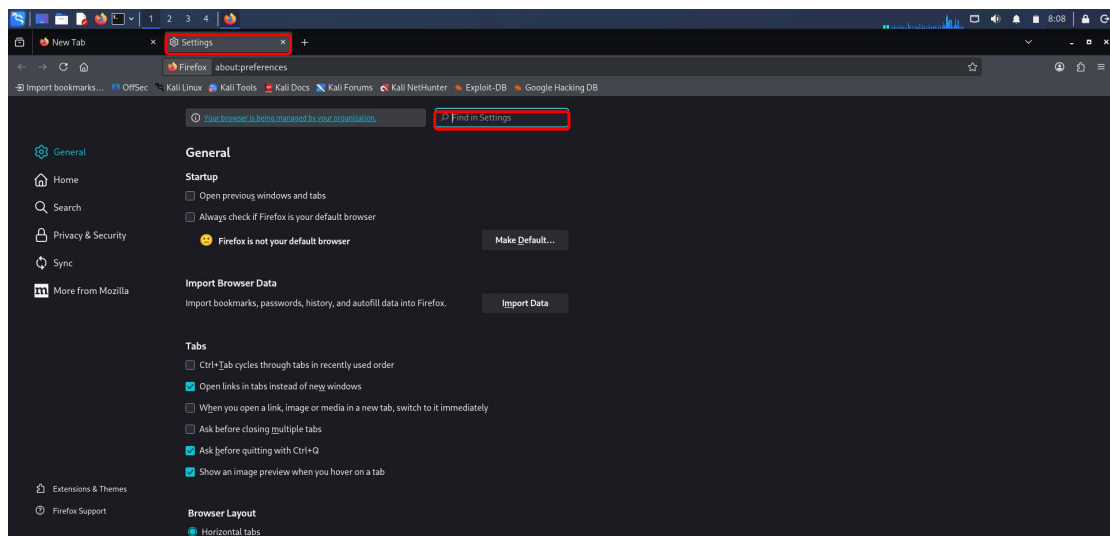
In above Picture a dashboard of Burpsuite is visible. Temporary project is created in Burp Suite because it's a limitation of community edition. Project is created with default setting of Burp suite.

9.5) Configuring FireFox Proxy

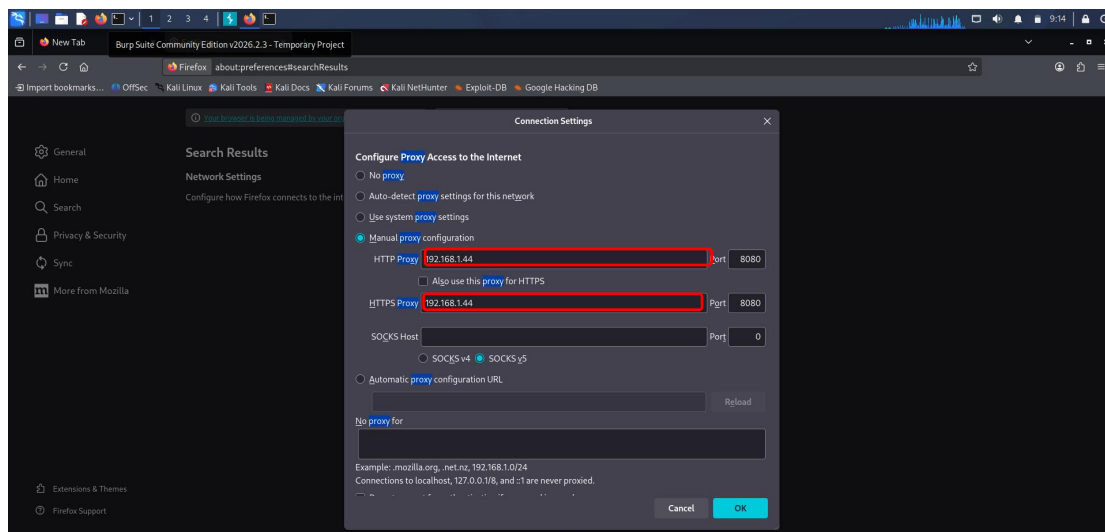
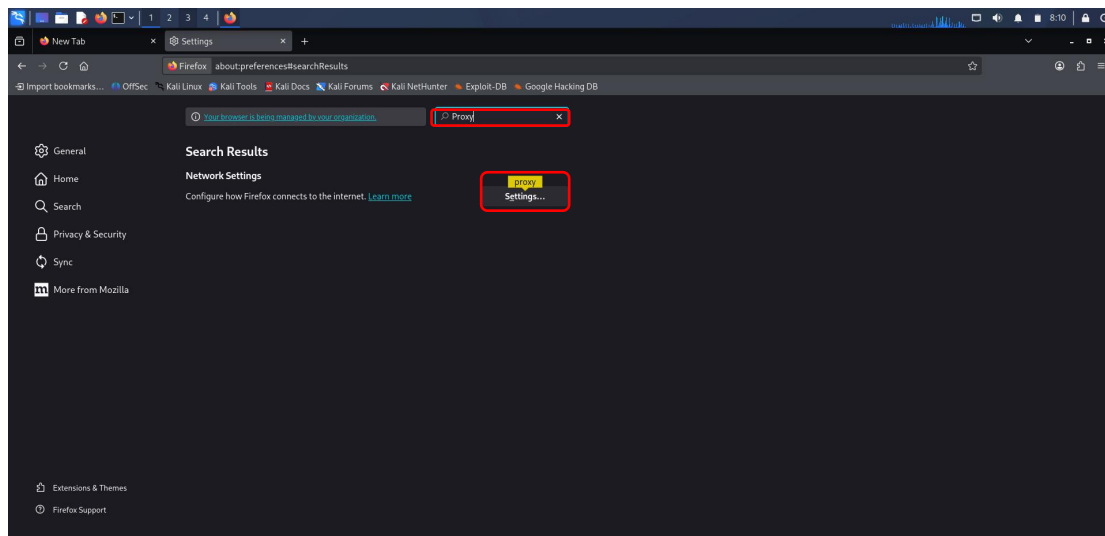
Open browser either google chrome or FireFox and then go to settings.



Search for Proxy in search bar.

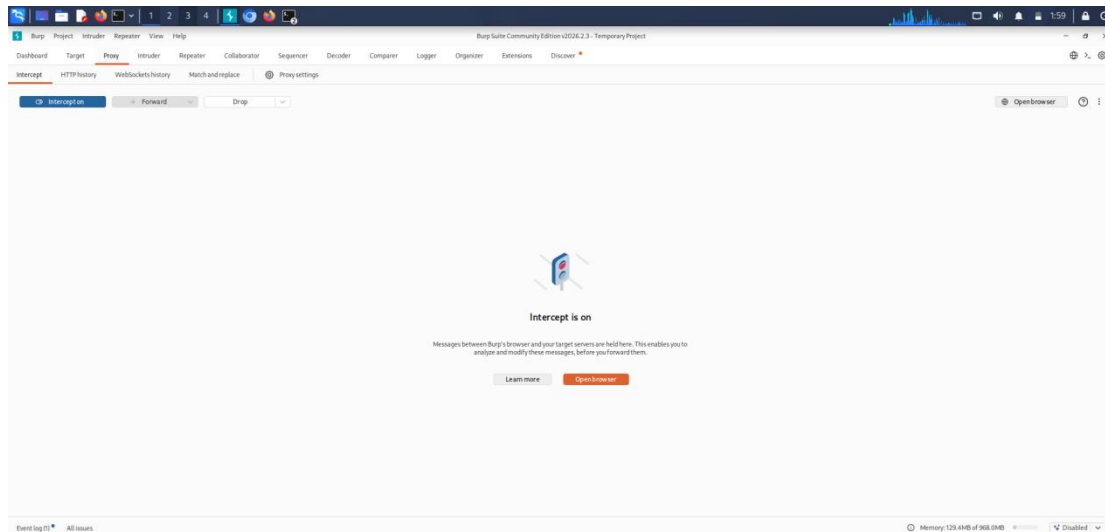


Click on Proxy Settings.



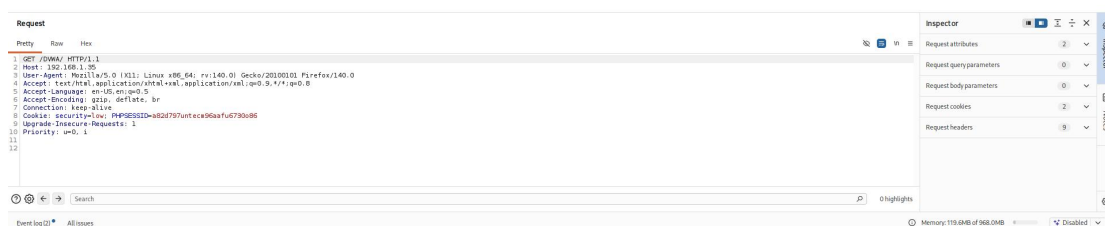
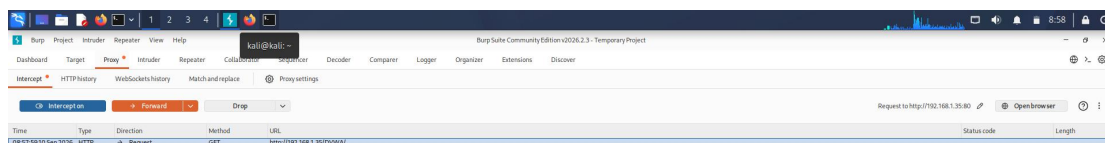
Change the Proxy settings write the Attacker machine IP address.

9.6) Enabling Burp suite Proxy



In the above figure The Burp Suite Intercept feature was enabled by navigating to **Proxy ---> Intercept**. This allows Burp suite to pause incoming HTTP requests before they reach the web server

9.7) Intercepting HTTP Request



As in the above figure it is clearly shown that request is intercepted for DVWA application. The Request contains following headers mentioned below.

Request Method: GET

Host: 192.168.1.35

User-Agent: Mozilla FireFox

Accept: text/html,application/xhtml+xml,application/xml;q=0.9, */*;q=0.8

Accept-Language: en-US

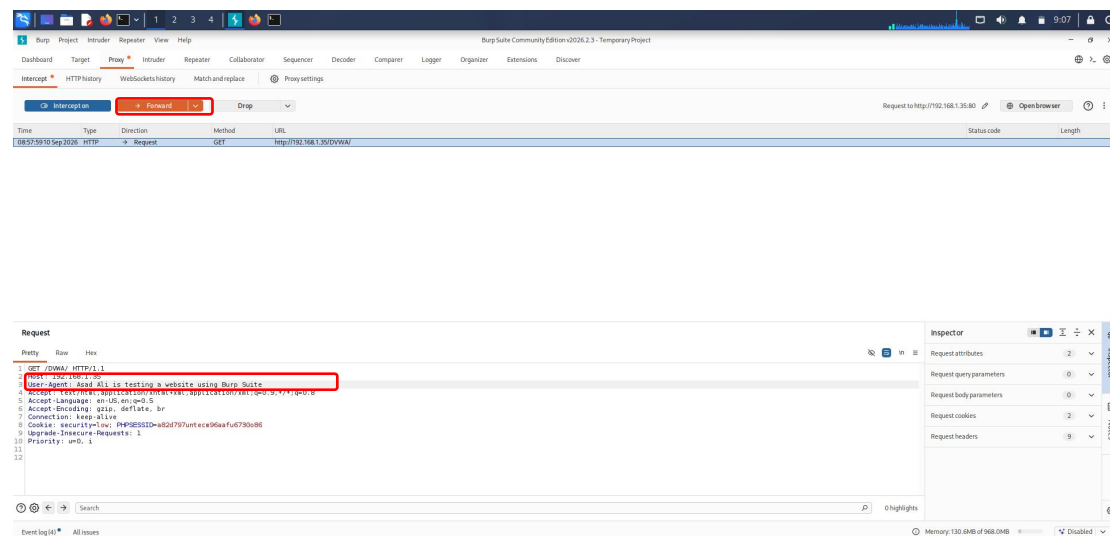
Accept-Encoding: gzip, deflate, br

Connection: keep-alive

Cookie: security=low, PHPSESSID=a82d797untecm96aafu6730o86

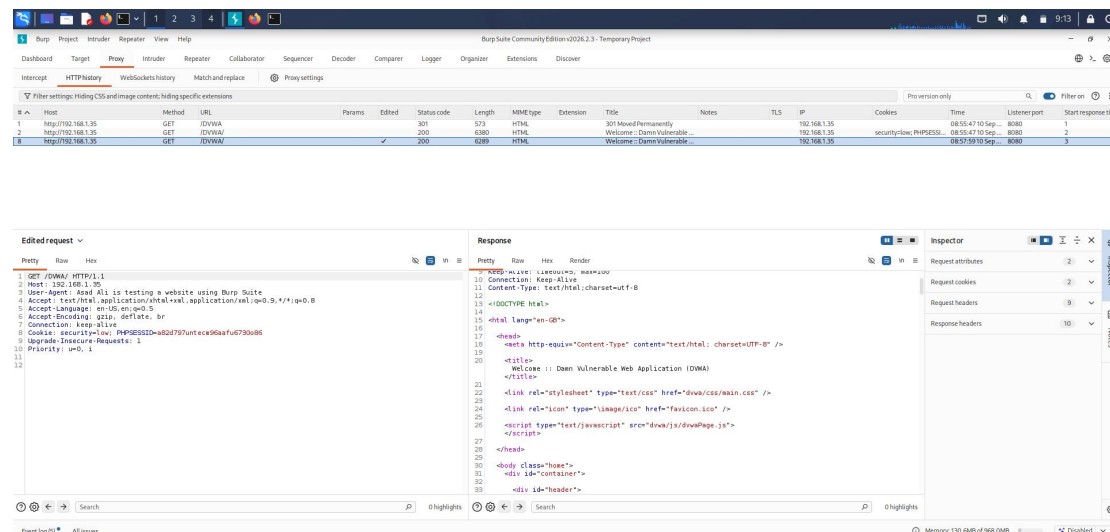
Upgrade-Insecure-Requests:1 Priority:u=0

9.7) Modifying HTTP Request



In the above Figure it I shown that Request is modified in the User agent and the modification statement is **Asad Ali is testing a website using Burp Suite**. After testing forward the request by clicking the forward button.

Verifying the Editing Request gets Response



On the left side in bottom there is EditRequest Section that has edited Request and on the right side a response section having response of the request.

10) Conclusion

The task successfully established a strong foundation in web application fundamentals and security testing concepts. Through the study of HTTP/HTTPS, requests and responses, HTTP methods, status codes, cookies, and sessions, a clear understanding of how web applications communicate was developed. Practical exposure to Burp Suite further strengthened this knowledge by configuring the tool, intercepting requests from the DVWA environment, modifying request parameters, and forwarding the modified requests. The task also provided an understanding of the major Burp Suite modules, including Proxy, Intercept, Intruder, Repeater, and Decoder, and their roles in web application security testing. Overall, the task provided both theoretical knowledge and controlled practical experience that serves as a foundation for further learning and performing web application security assessments.